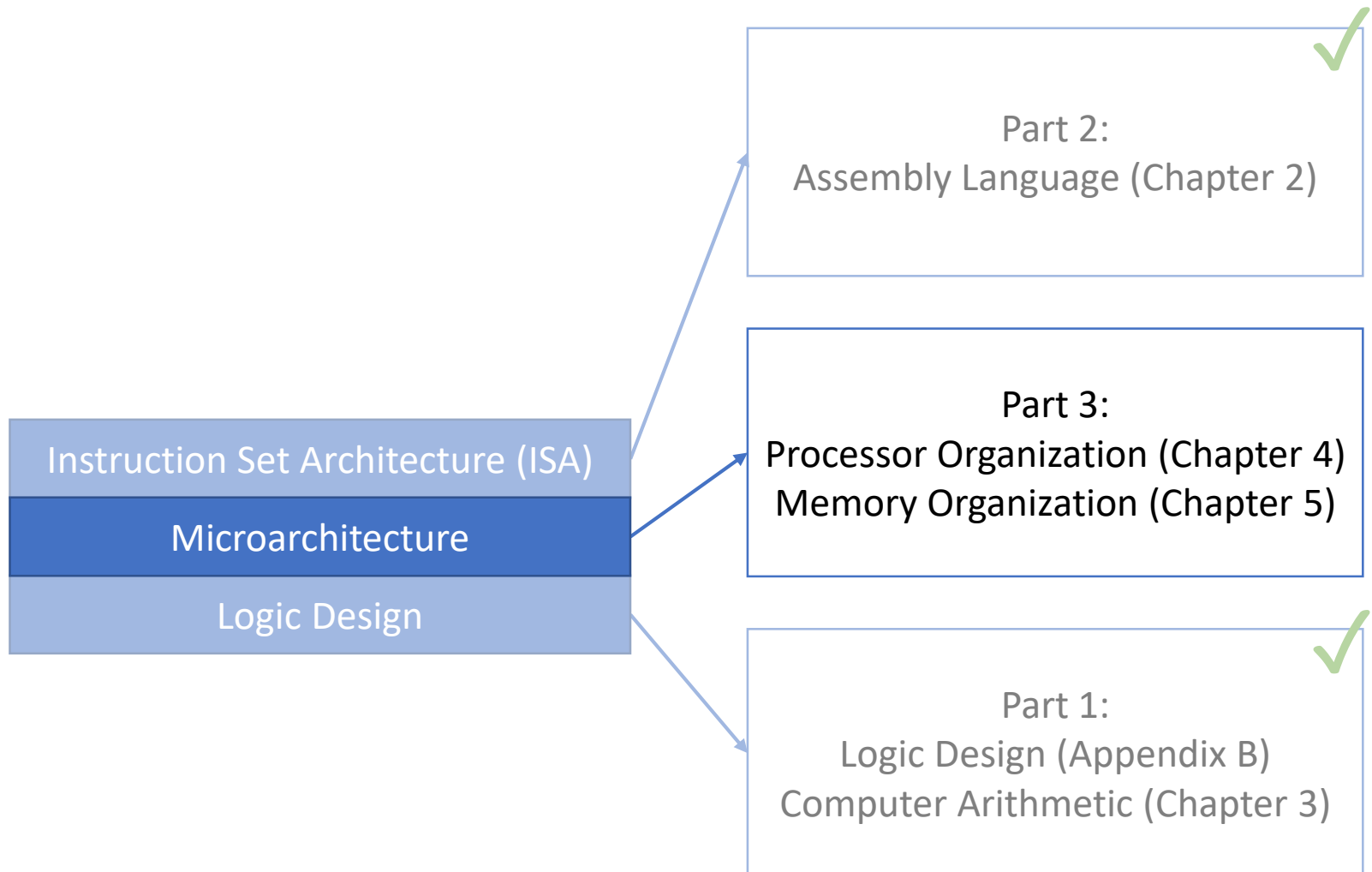


Lecture 31:

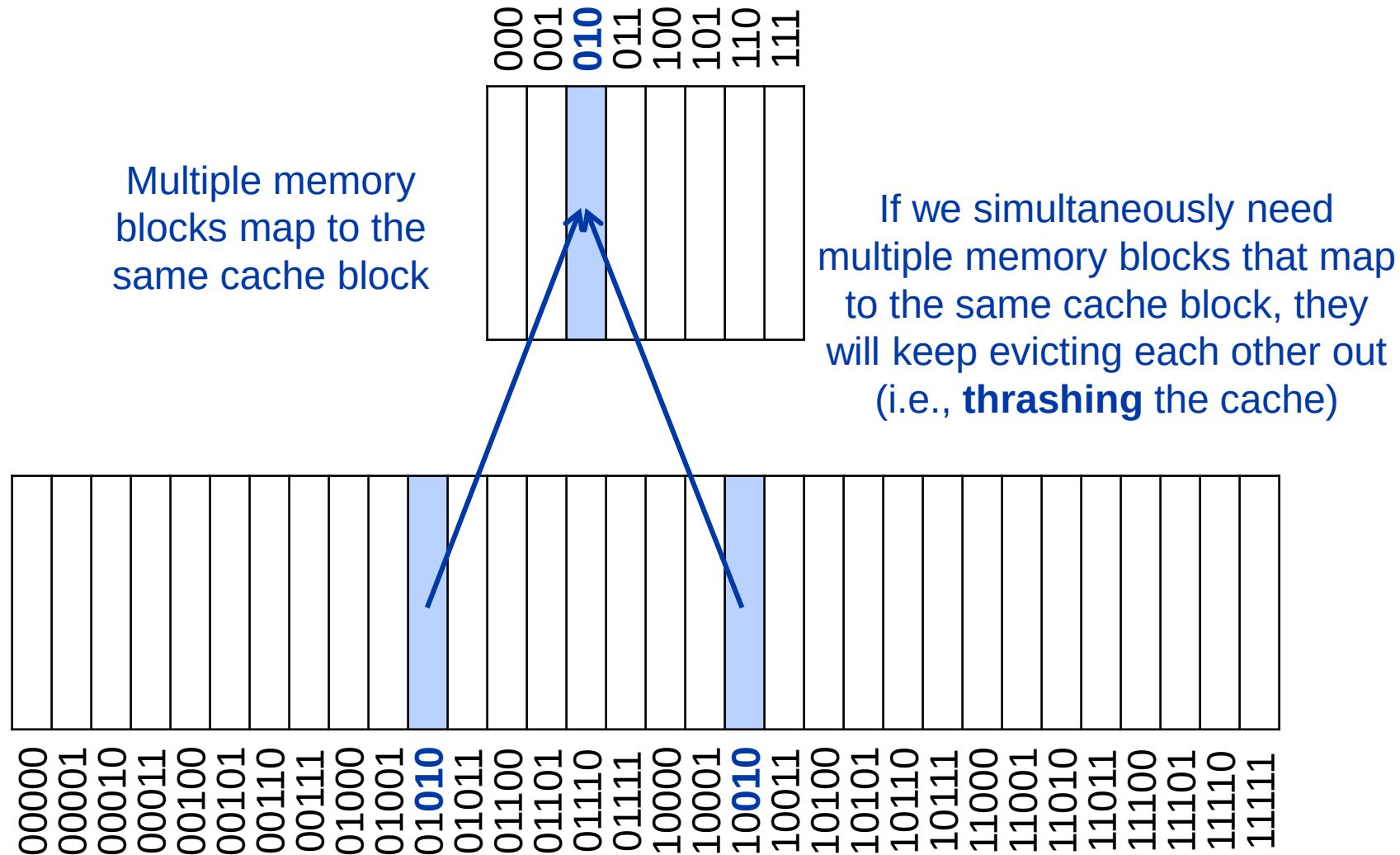
Cache Performance

CMPS 221 – Computer Organization and Design

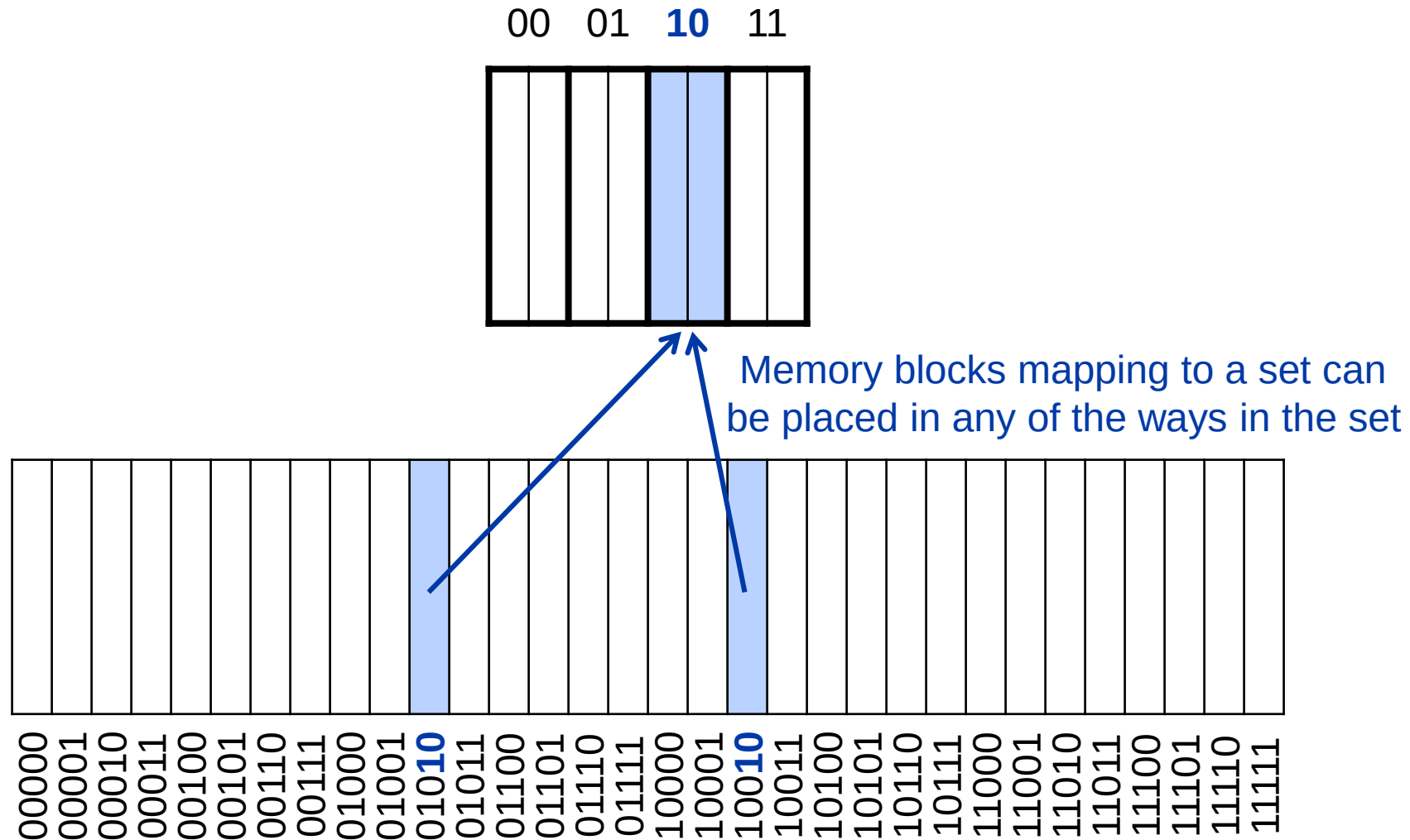
Last Time: Associative Caches



Recall: Direct Mapped Cache



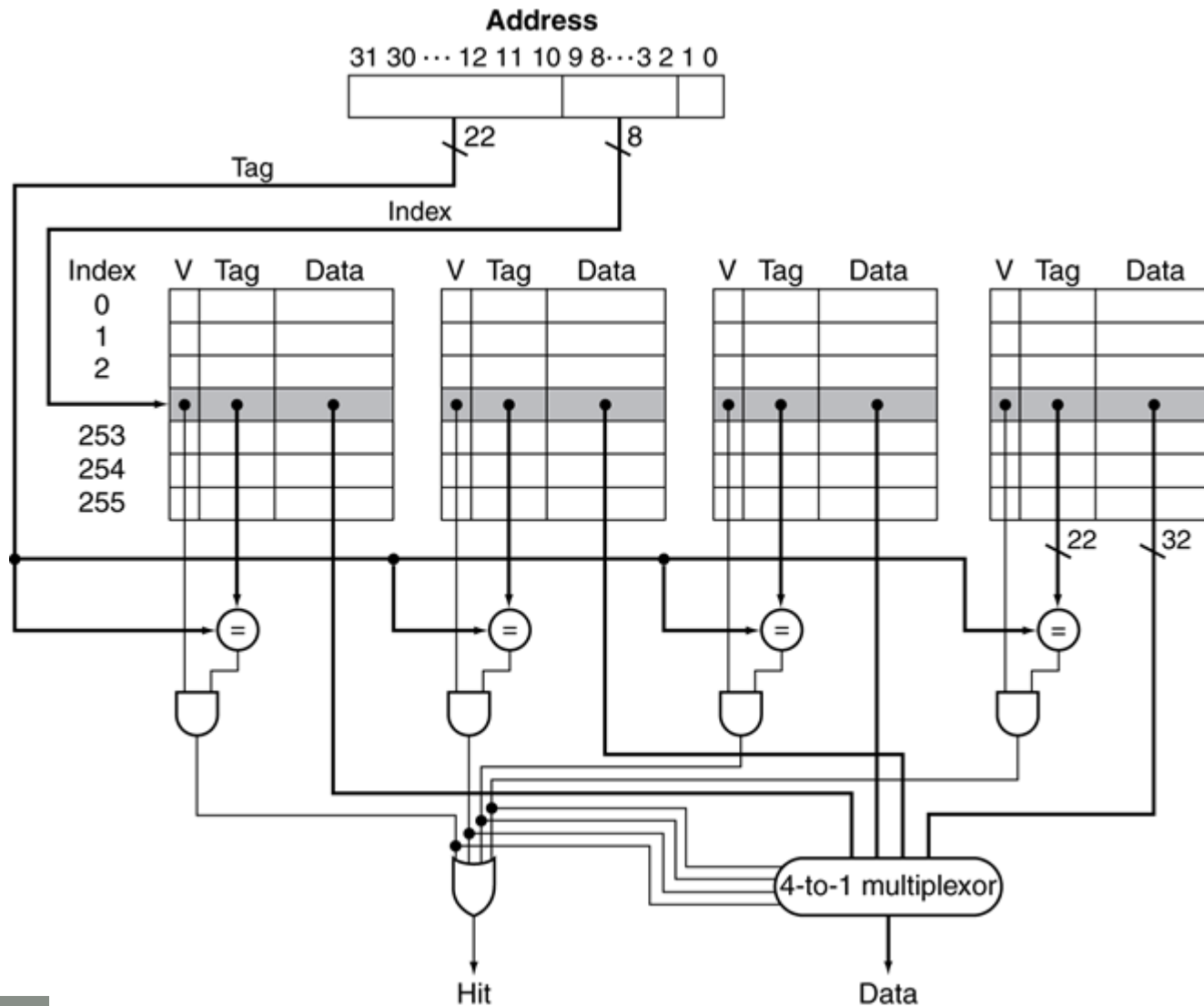
Associative Caches



Associative Caches

- **N -way set associative cache**
 - Each set contains N blocks
 - Search all entries in a given set at once
- **Fully associative cache**
 - A block can go in any cache entry
 - The whole cache is one set
 - Search all entries in the cache at once

Address Subdivision



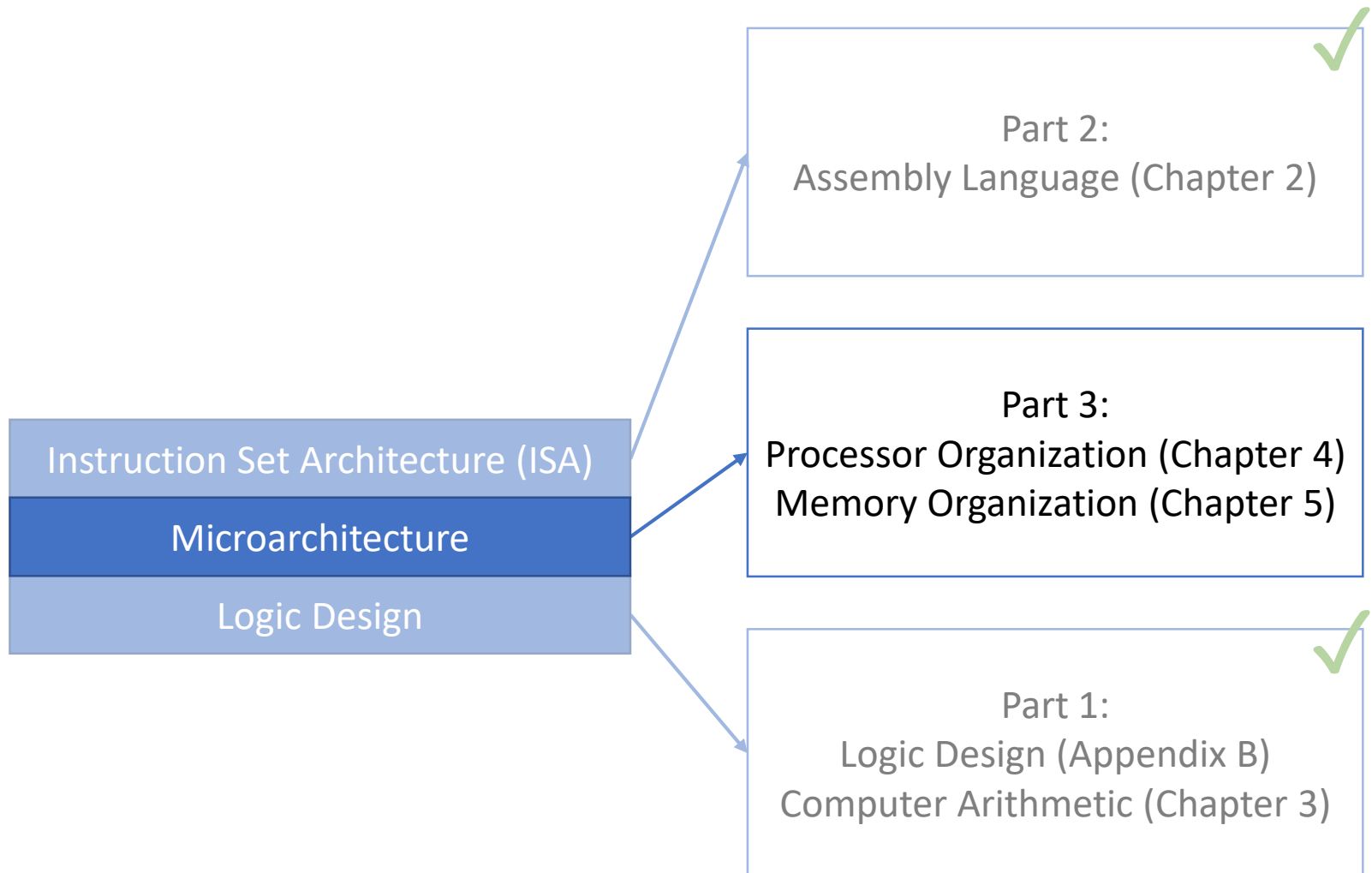
Replacement Policy

- A **replacement policy** selects which block to evict if there is no empty block available
- Direct mapped cache: only one choice
- Set associative cache
 - **Least-recently used** (LRU)
 - Choose the one unused for the longest time
 - Simple for 2-way, manageable for 4-way, too hard beyond that
 - **Random**
 - Gives approximately the same performance as LRU for high associativity

How Much Associativity

- Advantage of increasing associativity
 - Decreases miss rate
 - But with diminishing returns
 - Example: Simulation of a system with 64KB D-cache, 16-word blocks, SPEC2000 benchmark suite
 - 1-way: 10.3%
 - 2-way: 8.6%
 - 4-way: 8.3%
 - 8-way: 8.1%
- Disadvantage of increasing associativity
 - Increases hit time
 - More hardware (comparators, mux, etc.)

Today: Cache Performance



Sources of Hits

- **Temporal locality**

- Data is in the cache because it was previously accessed

- **Spatial locality**

- Data is in the cache because nearby data in the same block was recently accessed

Sources of Misses

- **Compulsory miss** (aka **cold start miss**)
 - Data is accessed for the first time
- **Capacity miss**
 - Data was previously accessed but was evicted due to finite cache size
- **Conflict miss** (aka **collision miss**)
 - Data was previously accessed but was evicted due to competition for blocks in the same set
 - Only happens in a non-fully-associative cache
 - Would not occur in a fully associative cache of the same size

Average Access Time

- Average memory access time (AMAT)

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

- Example:
 - Given the following:
 - **Hit time** = 1 cycle
 - **Miss rate** = 5%
 - **Miss penalty** = 20 cycles
 - $\text{AMAT} = 1 + 0.05 \times 20 = 2 \text{ cycles}$

Minimizing Access Time

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

- To minimize access time:
 - Minimize **hit time**
 - Minimize **miss rate**
 - Minimize **miss penalty**
 - i.e., minimizing hit time and miss rate of next level

Hit Time vs. Miss Rate

- Tension between **hit time** and **miss rate**
 - Minimizing **hit time** favors:
 - Small block size
 - Small cache size
 - Low associativity
 - Minimizing **miss rate** favors:
 - Large block size (minimizes compulsory misses)
 - Large cache size (minimizes capacity misses)
 - High associativity (minimizes conflict misses)
- Solution: multilevel caches

Multilevel Caches

- Level-1 (L1) cache
 - Closest to CPU (accessed in IF/MEM stages)
 - Focus on **low hit time** to minimize IF/MEM duration
 - Small and fast
- Level-2 (L2) cache
 - Services misses from L1 cache
 - Focus on **low miss rate** to avoid DRAM access
 - Larger and slower, but still faster than DRAM
- Main memory services L2 cache misses
 - Some high-end systems include L3 cache in between L2 and main memory

AMAT for Multi-level Caches

- The miss penalty is the AMAT of the next level of the hierarchy
- Given the following:
 - L1 hit time = h_1 , L1 miss rate = m_1
 - L2 hit time = h_2 , L2 miss rate = m_2
- AMAT across the hierarchy
 - $$\begin{aligned} \text{AMAT}_{L1} &= h_1 + m_1 \times \text{AMAT}_{L2} \\ &= h_1 + m_1 \times (h_2 + m_2 \times \text{AMAT}_{\text{DRAM}}) \end{aligned}$$

Impact on CPI

- Components of CPU time:
 - Program execution cycles
 - Includes cache hit time
 - Memory stall cycles
 - Mainly from cache misses
 - Instruction cache (I-cache) misses stall IF stage
 - Data cache (D-cache) misses stall MEM stage

Impact on CPI

- Given:

- CPI for ideal cache (no misses) = CPI_0
- # of instructions = I
- Fraction of memory instructions = f
- L1 I-cache miss rate = m_{1i}
- L1 D-cache miss rate = m_{1d}
- L1 miss penalty = t

- With simplifying assumptions:

- I-cache stalls = $I \times m_{1i} \times t$
- D-cache stalls = $I \times f \times m_{1d} \times t$
- cycles = $I \times CPI_0 + I \times m_{1i} \times t + I \times f \times m_{1d} \times t$
- $CPI = CPI_0 + m_{1i} \times t + f \times m_{1d} \times t$

Textbook Sections

- The content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - 5.4